LIDR

Módulo 5 · Los prompts de la sesión

Todo lo que se teclea hoy, en orden, para que puedas seguir la sesión sin copiar del chat y para que puedas repetirla después en tu propio proyecto, que es donde de verdad sirve.

No es el guion del mentor. Aquí está lo que se ejecuta y **por qué cada restricción está donde está**, que es la parte que se pierde cuando alguien copia un prompt suelto.

Antes de empezar

Trabaja sobre **tu propio fork** de:

`https://github.com/LIDR-academy/flowsync-ai4devs`

Fork, no clone directo: durante la sesión se abre un cambio propuesto, y sobre un clon del repositorio del curso no tienes permiso de escritura. La secuencia es **fork** → **clone de tu fork** → `git checkout -b s5/start upstream/s5/start` → **instalar dependencias**, y está detallada en el prework del módulo.

> 💻 **Todo se teclea desde la raíz del repo.** Los comandos de terminal van entre paréntesis, `(cd backend && …)`, para que funcionen estés donde estés y te devuelvan a la raíz. Este proyecto no tiene `package.json` en la raíz: `npm` se ejecuta dentro de `backend/` o de `frontend/`, y eso lo dice su propio `CLAUDE.md`.

Ver el trabajo que hay hecho

En la terminal, desde la raíz del repo:

```
git log --oneline -15
```

Quince commits de trabajo real, hechos con tu harness puesto. Y al lado, en `CLAUDE.md`, las reglas de proceso que ese trabajo tenía que cumplir. **Nadie ha comparado nunca las dos cosas**, y de eso va la primera demo.

Auditar tus propias reglas

```
Lee las reglas de proceso de `CLAUDE.md` y contrástalas contra los últimos 15 commits de este repositorio y el estado actual del árbol. Para cada regla, una fila: qué dice, qué evidencia has mirado, y si se ha cumplido, no se ha cumplido, o no se puede comprobar desde el historial. Usa solo lo que puedas verificar con comandos de git y leyendo archivos: si una regla no deja rastro comprobable, dilo en vez de suponer. No cambies ningún archivo: esto es una auditoría, no un arreglo.
```

Las tres restricciones hacen falta. Pedir **la evidencia de cada fila** es lo que convierte una opinión en algo que puedes discutir señalando el historial. **"Si no deja rastro, dilo"** es la más importante: sin ella el agente rellena, y la mitad de las reglas de un harness no dejan ninguna huella en git. Y **"no cambies ningún archivo"** evita que se ponga a arreglarte el harness antes de que hayas visto qué estaba roto.

⚠️ **No hay un resultado "correcto" aquí.** Puede salir que alguna regla no se cumple, que varias no se pueden comprobar, o que se cumplen todas. Los tres casos dicen algo. El tercero no significa que estés a salvo: significa que **has tenido suerte, no garantía**, porque nada lo estaba comprobando.

> 💡 **Este es el ejercicio que más rinde de todo el módulo, y no cuesta nada repetirlo.** Hazlo en un proyecto tuyo de verdad esta semana. Lo normal es encontrar tres grupos: reglas que se cumplen solas y que puedes borrar, reglas que no se cumplen y a nadie le ha importado nunca (bórralas también, son ruido con autoridad), y una o dos que no se cumplen y que explican algo que llevabais tiempo achacando a otra cosa. Ese último grupo es tu lista de qué automatizar, y la has obtenido midiendo.

Bajar una regla al pipeline

Primero la comprobación, en local:

```
Hoy el contrato de la API de este proyecto solo existe mientras el servidor corre: se construye en cada petición y se sirve por HTTP, y no hay ningún fichero en el repositorio. Haz dos cosas. Primero, añade un comando del proyecto que genere ese documento y lo escriba en un fichero versionado dentro de `docs/`. Segundo, añade una comprobación que regenere el documento en una ubicación temporal, lo compare con el versionado, y termine con código de salida distinto de cero explicando la diferencia si no coinciden. Llama `openapi:generate` al comando que genera y `openapi:check` a la comprobación, expón los dos como scripts del proyecto con esos mismos nombres, y escribe el documento en `docs/api/openapi.json`. La comprobación solo compara, no arregla: no toques las rutas ni dejes que reescriba el fichero versionado.
```

Fíjate en que el prompt pide **dos cosas y en ese orden**, porque ahí está la lección: hoy ese documento **no se puede vigilar**, ya que no existe como fichero. Solo se construye cuando el servidor responde. Y un artefacto que solo existe en tiempo de ejecución no se puede diffear, ni revisar en un cambio propuesto, ni comparar con nada: **no admite guardarraíl**. Así que el primer trabajo no es vigilarlo, es materializarlo.

Del resto: **"código de salida distinto de cero"** es lo que lo convierte en un guardarraíl en vez de en un informe, porque es lo único que un sistema de integración continua sabe leer. **"En una ubicación temporal"** evita un error clásico y silencioso, que la comprobación reescriba el fichero que está vigilando. Y **"solo compara, no arregla"** mantiene separado quien vigila de quien repara.

Después, en la terminal, desde la raíz del repo:

```
(cd backend && npm run openapi:check)
```

Sale verde. **Y eso todavía no demuestra nada**, que es el punto entero de esta demo.

> 🧯 **Este minuto es el que hace el ejercicio, no te lo saltes.** Abre `backend/start/routes.ts`, renombra un endpoint (por ejemplo `profile` por `perfil`), guarda y **no regeneres nada**. Vuelve a correr el comando: tiene que salir **rojo**, explicando la diferencia. Después revierte con `git checkout -- backend/start/routes.ts` y vuelve a correrlo: verde.
>
> Una comprobación que solo has visto en verde podría estar comparando el fichero consigo mismo, o no comparando nada. Es el mismo problema del semáforo verde, aplicado a la herramienta que acabas de escribir.
>
> Y fíjate en que el caso que acabas de provocar **es el real**: nadie edita a mano un fichero generado. Lo que pasa de verdad es que alguien cambia el código y se olvida del artefacto que colgaba de él. *(De propina: con esa ruta cambiada, la suite de tests también se pone roja, porque consume los tipos que salen de esas mismas rutas. Es la jerarquía de la lección: lo que se puede responder ejecutando, se responde ejecutando.)*

Y ahora al pipeline, que es donde vale para todo el equipo y no solo para tu máquina:

```
Crea un trabajo de integración continua en `.github/workflows/` que ejecute esa comprobación sobre cada cambio propuesto y sobre cada push a la rama principal. Que instale las dependencias del backend y corra el script. Dale al trabajo únicamente los permisos que necesite y nada más, y pon un tiempo máximo de ejecución. Explícame en dos líneas qué hará y qué no.
```

"Únicamente los permisos que necesite" es higiene que el agente no aplica solo, y en un archivo que corre con acceso a tu repositorio importa. **"Un tiempo máximo"** impide que una ejecución colgada consuma minutos toda la noche. Y **"qué hará y qué no"** te obliga a leer el límite en vez de suponerlo.

El revisor que llega solo

```
Crea otro trabajo de integración continua en `.github/workflows/` que lance un revisor de código sobre cada cambio propuesto, usando la acción oficial de Claude Code para GitHub Actions. Que reciba su credencial desde un secreto del repositorio, y que publique los hallazgos como comentarios en el cambio propuesto en vez de dejarlos solo en el registro de ejecución. Dale herramientas de lectura sobre el repositorio (Read, Grep y Glob en su lista de herramientas permitidas): sin ellas no puede abrir ni el archivo de calibración ni un solo archivo del cambio, y cada intento denegado le cuesta un turno. Dile además que no redirija salida a ficheros, que lea la salida directa. Que use un modelo económico (--model sonnet --effort medium), acote el trabajo a 40 turnos (--max-turns 40) y el reloj a 10 minutos (timeout-minutes: 10), con esos valores exactos. Debajo del archivo, escríbeme en `docs/ci-revisor.md` qué pasos manuales quedan pendientes para que esto funcione de verdad, y en qué se va el dinero: qué modelo usa el revisor, por qué ese y cómo cambiarlo. Para el precio, enlaza la página oficial en vez de citar cifras: cambian, y una cifra escrita aquí envejece sin que nadie se entere. Sobre la credencial, documenta dos caminos y pon primero el de suscripción: el token que se genera en local con `claude setup-token`, y solo después la clave de API de la consola. Di de cada uno qué entrada de la acción usa y con qué nombre se guarda el secreto, porque no son intercambiables, y avisa de que cruzarlos falla en la primera llamada sin explicar por qué. Incluye también qué ocurre con los cambios propuestos desde un fork en un repositorio público.
```

Las cuatro piezas importan, y la primera es la que más gente falla: **por defecto los hallazgos se quedan en el registro de ejecución del proceso**, que es un log al que hay que entrar a buscar. Nadie hace eso. Un hallazgo que vive ahí, en la práctica, no existe. Hay que pedir explícitamente que se publiquen donde la gente mira.

Las **herramientas de lectura** son la pieza sin la que nada funciona: sin ellas el revisor no puede abrir ni su propio archivo de calibración. El **límite de turnos y el tiempo máximo** son las palancas de coste. Y lo de los pasos pendientes es lo honesto: este archivo **no funciona por estar escrito**.

> 🔑 **Las dos credenciales, porque se confunden y el fallo no avisa.**
>
> | de dónde sale | entrada de la acción | nombre del secreto |
> |---|---|---|
> | `claude setup-token` (suscripción Pro o Max) | `claude_code_oauth_token` | `CLAUDE_CODE_OAUTH_TOKEN` |
> | `console.anthropic.com` → API Keys | `anthropic_api_key` | `ANTHROPIC_API_KEY` |
>
> Los dos funcionan. **Cruzados no**, y cuando fallan lo hacen en la primera llamada, sin gastar nada y **sin decir en ningún sitio que el problema es el nombre**. Con el camino de suscripción el saldo de API no se mueve, porque el consumo va contra tu plan: **no es gratis, se cobra en otro sitio**.

> ⚠️ **Y un límite que no arregla ninguna configuración, que te toca a ti directamente.** En un repositorio público, **un cambio propuesto desde un fork no recibe los secretos**. Es una protección correcta: si los recibiera, cualquiera podría proponer un cambio que imprima la clave y leerla en el registro.
>
> **Tú trabajas sobre tu fork, así que tu revisor no va a correr sobre tu propio cambio propuesto.** No está roto: está funcionando como debe. En la sesión lo verás correr sobre el repositorio del curso, y la consecuencia que importa es la de fondo — **el revisor automático no cubre las contribuciones que llegan de fuera**, que son justo las que más falta hace revisar.

Por eso se ejecuta además **su mismo encargo** en local, para ver qué produce sin depender de nada de esto:

> ```
> Ejecuta ahora, en local, el mismo encargo que le hemos dado a ese revisor automático: pasa el subagente `adversarial-reviewer` sobre la capability de tareas, contrastando contra los scenarios de `openspec/specs/tasks/spec.md`. No arregles nada: hoy solo queremos ver el informe que habría aparecido en el cambio propuesto.
> ```

⏱️ **Es el paso más lento de la sesión.** Es normal que tarde varios minutos.

Fíjate en lo que aparece: los dos defectos que te llevaste de deberes del módulo anterior, **sin que nadie se los haya señalado**. Nadie ha dicho "mira el modelo de tareas". Ha llegado, ha leído la especificación, y ha vuelto con ellos. Esa es toda la diferencia entre pedirle a un agente que revise tu código y que un agente revise tu código.

Y antes de que se convierta en ruido, se calibra:

```
Escribe un archivo de instrucciones solo para revisiones, en la raíz del repositorio, que calibre a ese revisor para este proyecto. Cuatro cosas: define qué cuenta como hallazgo grave aquí y qué no pasa de sugerencia; pon un tope al número de sugerencias menores por revisión y di que el resto se resuma contadas; lista dónde no debe reportar, incluyendo cualquier cosa que otra comprobación del repositorio ya vigile; y exige que para afirmar que algo se comporta de una manera cite el archivo y la línea donde lo ha visto, en vez de deducirlo del nombre. Que quepa en una pantalla: un archivo largo diluye las reglas que importan.
```

De las cuatro, **el tope de sugerencias menores es la más eficaz y la que casi nadie pone**. Nadie actúa sobre veinte comentarios; casi todo el mundo actúa sobre cinco. Un revisor que deja quince comentarios correctos en cada cambio se lee la primera semana, se hojea la segunda, y al mes se cierra en bloque sin abrirlo. El día que uno de ellos señale algo grave, se cerrará igual.

Cerrar el trabajo del día

```
Ya está todo el trabajo de hoy en esta rama. Abre el PR con `gh pr create` y escribe una descripción que cuente la historia completa: qué reglas se auditaron y cuál no se cumplía, qué comprobación determinista se añadió y qué protege, qué trabajo de integración continua queda escrito y qué le falta para funcionar, y cómo se ha calibrado el revisor. No abras más de uno.
```

Lo que te llevas

- Una **auditoría** de tus propias reglas de proceso, con la evidencia de cada una.
- Una **comprobación determinista** que verifica que el contrato versionado sigue coincidiendo con el código, **y que has visto fallar a propósito**.
- **Dos trabajos de integración continua** escritos, con sus pendientes documentados sin adornos.
- Un **archivo de calibración** que acota qué es grave y cuántas sugerencias menores caben.
- Un **informe de revisión** que cazó dos defectos que nadie le señaló.

> **Y la frase para llevarse del bloque entero:** lo que no se comprueba, no está garantizado. Da igual lo bien que lo hayas escrito, y da igual en qué archivo lo hayas puesto.